



AutoQA Game Evaluation Metrics Method

목적

AutoQA의 목표는 게임에서 수집되는 시스템 데이터와 관측 데이터를 근거로 자동 평가 지표를 생성하고, LLM 기반 영상 분석까지 결합해 사람이 반복하던 QA 판정을 재현 가능한 지표 체계로 바꾸는 것이다.

연구 목표는 초기 버전에서 검증 지표 10개 이상을 정의하고, 각 지표가 어떤 관측 데이터와 검색 근거, 영상 분석 결과로 계산되는지 명확히 고정하는 것이다.

설계 관점

이 문서는 `ooo` 관점에서는 먼저 평가 계약을 고정하는 `seed` 문서 역할을 한다. 지표 이름만 나열하지 않고 입력, 계산식, 합격 기준, 검증 방법을 함께 적어 이후 구현이 문서와 drift되지 않게 한다.

`autoresearch` 관점에서는 실제 Karpathy식 `train.py` 탐색 루프를 실행하지 않는다. 대신 그 핵심 원칙만 AutoQA에 맞게 차용한다.

- 사람이 `metric_program.md` 에 연구 목표와 제약을 작성한다.
- 평가 데이터셋과 판정 기준은 세션 중 바꾸지 않는다.
- 후보 지표는 하나씩 추가하거나 수정한다.
- 결과는 append-only로 기록한다.
- 사람 판정과의 일치율, 결함 발견률, 재현성을 개선한 지표만 유지한다.

현재 프로젝트와의 연결

FunQA는 검색을 단순 채팅이 아니라 검증 가능한 retrieval workspace로 다룬다. 기존 계약인 `graph-core retrieval`, `document-graph consensus`, `evidence-only fallback` 을 AutoQA 에도 그대로 적용한다.

AutoQA에서는 다음과 같이 대응한다.

FunQA 개념	AutoQA 대응
문서/청크 검색	게임 기획서, QA 체크리스트, 패치 노트, 버그 리포트, 튜토리얼 문서 검색
그래프 검색	게임 객체, UI 요소, 퀘스트, 상태, 액션, 실패 조건 간 관계 검색
consensus gate	관측 데이터, 검색 근거, 영상 판독 결과가 같은 결론인지 판단
evidence-only fallback	합의가 부족하면 자동 합격/불합격 대신 근거 묶음만 반환
rag-lab inspect	지표별 입력, 검색 근거, LLM 판정, 실패 이유를 디버깅하는 AutoQA lab

입력 데이터 계약

스크린샷의 관측 데이터는 AutoQA의 1차 입력 계약으로 둔다.

관측 데이터	세부 필드	AutoQA 용도
Image Coordinate	좌표 위치 정보, bounding box 정보	UI/캐릭터/오브젝트 위치 정확도, 시야 내 노출, 화면 구성 검증
Log	탐지된 이미지 개수, 시간, FPS, 상태	이벤트 타임라인, 성능, 상태 전이 검증
Macro	mouse, key, 작업 큐, 프로세스 핸들링	입력 재현성, 조작 반응성, 자동 플레이 경로 검증
OCR	텍스트 출력, 좌표 위치 정보, bounding box 정보	UI 문구, 퀘스트 텍스트, 에러 메시지, 로컬라이제이션 검증
Object Detecting	object 좌표 위치 정보, bounding box 정확도, 신뢰도	게임 객체 존재 여부, 충돌/상호작용 대상, 장면 상태 검증

추가로 영상 분석 자동화를 위해 다음 파생 입력을 둔다.

- videoSegment : 시작/종료 timestamp, 장면 ID, 샘플링 FPS
- frameObservation : 프레임 timestamp, OCR 결과, 객체 검출 결과, 주요 좌표
- audioTranscript : 대사, 효과음 라벨, timestamp

- `ragEvidence` : 검색된 문서, 그래프 경로, citation
- `llmJudgement` : 구조화된 판정 JSON, 근거 timestamp, confidence

지표 생성 방법

1. 검색 기반 후보 지표 생성

1. 게임 문서와 QA 자료를 FunQA 검색엔진에 적재한다.
2. 문서를 `objective`, `ui_element`, `game_object`, `state`, `action`, `failure_condition`, `expected_result` 로 추출한다.
3. 그래프 노드는 퀘스트, 레벨, 캐릭터, 아이템, UI, 입력, 상태, 로그 이벤트 로 구성한다.
4. 검색 질의 템플릿으로 평가 후보를 만든다.

예시 질의:

이 게임에서 플레이어 진행 상태를 자동 검증할 수 있는 화면/로그/입력 기반 조건은 무엇인가?
 이 UI 요소가 정상 표시되었는지 확인하는 데 필요한 OCR과 좌표 조건은 무엇인가?
 이 보스전에서 실패로 간주해야 하는 로그 이벤트와 영상 이벤트는 무엇인가?

검색 결과는 바로 지표가 되지 않는다. AutoQA는 각 후보를 아래 스키마로 변환한 뒤 consensus gate를 통과한 것만 지표 목록에 넣는다.

```
{
  "metricId": "autoqa.ui.coordinate_consistency",
  "name": "UI Coordinate Consistency",
  "inputSignals": ["imageCoordinate", "ocr", "objectDetection"],
  "referenceEvidence": ["design-doc#ui-hud", "qa-checklist#hud-visible"],
  "calculation": "observed_bbox overlaps expected_region by IoU >= threshold",
  "passCriteria": "score >= 0.90 for required UI elements",
  "fallback": "evidence-only when expected region is missing from retrieved docs"
}
```

2. LLM 영상 분석 기반 후보 지표 생성

LLM 영상 분석은 원본 영상을 직접 보거나, 프레임 샘플링과 OCR/object 결과를 함께 넣어 구조화된 관측 결과를 만든다. Gemini 공식 문서는 비디오에서 설명, 정보 추출, timestamp 기반 질의응답을 지원하고,

기본 시각 샘플링은 1 FPS라 빠른 액션은 커스텀 프레임레이트가 필요하다고 설명한다. OpenAI cookbook의 비디오 이해 예시는 프레임을 추출해 비전 모델에 입력하는 우회 방식을 제시한다.

AutoQA에서는 두 경로를 모두 허용한다.

- Native video path: Gemini처럼 video input을 받는 모델에 영상, 오디오, timestamp 질의를 함께 전달한다.
- Frame path: OpenCV 등으로 프레임을 추출하고, OCR/object/macro/log와 함께 LLM에 전달한다.

LLM 출력은 반드시 JSON으로 제한한다.

```
{
  "segmentId": "stage1-boss-001",
  "events": [
    {
      "timestamp": "00:12.4",
      "eventType": "player_hit",
      "evidence": {
        "visual": "damage flash and HP decrease",
        "ocr": "HP 72 -> 51",
        "log": "DamageApplied"
      },
      "confidence": 0.86
    }
  ],
  "metricCandidates": [
    {
      "metricId": "autoqa.combat.damage_feedback_alignment",
      "reason": "visual damage feedback, OCR HP delta, and log event can be cross-checked"
    }
  ]
}
```

3. Consensus gate

자동 지표는 다음 세 근거 중 최소 두 개 이상이 같은 결론을 내고, 핵심 지표는 세 근거가 모두 맞을 때만 합격 판정한다.

근거	예시
관측 데이터	좌표, OCR, object confidence, log event, macro input
검색 근거	기획서, QA 체크리스트, 패치 노트, 기존 버그
LLM 영상 판독	timestamp 기반 이벤트, 장면 설명, 이상 징후 판정

합의가 부족하면 지표 점수를 확정하지 않고 `evidence-only` 로 반환한다. 이 원칙은 FunQA의 consensus-backed answer 정책과 같다.

초기 AutoQA 지표 14개

초기 연구 목표인 10개 이상을 만족하기 위해 14개를 제안한다. 각 지표는 이후 세션에서 데이터셋 기준으로 유지/폐기한다.

ID	지표	입력	계산 방법	합격 기준 예시
M01	UI Coordinate Consistency	Image Coordinate, OCR	필수 UI bbox와 기대 영역 IoU 평균	<code>meanIoU >= 0.90</code>
M02	OCR Text Fidelity	OCR, 검색 근거	기대 문구와 OCR 텍스트의 exact/semantic match	필수 문구 <code>100%</code> , 보조 문구 <code>>=0.95</code>
M03	Object Presence Recall	Object Detecting, 영상	장면별 필수 객체 탐지율	<code>recall >= 0.95</code>
M04	Object Localization Precision	Object Detecting, Image Coordinate	객체 bbox와 기준 bbox IoU	<code>IoU >= 0.75</code>
M05	Log-Visual Event Alignment	Log, 영상, Object Detecting	로그 이벤트와 영상 이벤트 timestamp 차이	<code>delta <= 500ms</code>

ID	지표	입력	계산 방법	합격 기준 예시
M06	Macro Replay Success Rate	Macro, Log, 영상	동일 매크로 실행 시 목표 상태 도달 비율	<code>successRate >= 0.98</code>
M07	Input Responsiveness	Macro, Log, 영상	입력 timestamp와 화면 반응 timestamp 차이	<code>p95 <= 150ms</code>
M08	State Transition Validity	Log, OCR, 검색 근거	상태 전이가 허용 그래프 경로와 일치하는지	불법 전이 <code>0건</code>
M09	Quest/Task Completion Evidence	OCR, Log, 영상, 검색 근거	완료 텍스트, 보상 로그, 상태 변화 합의	세 근거 중 <code>>=2</code> 합의
M10	Performance Stability	Log	FPS, frame drop, hitch count	p95 FPS 목표 이상, hitch budget 이하
M11	Error/Anomaly Detection	OCR, Log, 영상	에러 팝업, stuck, crash, 반복 루프 탐지	critical anomaly <code>0건</code>
M12	Tutorial Guidance Compliance	OCR, Macro, 영상	안내 문구와 허용 입력 순서 일치	단계별 순서 위반 <code>0건</code>
M13	LLM Video Judge Agreement	LLM 판정, 사람 라벨	사람 QA 판정과 LLM 구조화 판정 일치율	<code>agreement >= 0.90</code>
M14	Multimodal Consensus Confidence	전체 입력	관측/검색/영상 근거 간 합의 가중 평균	핵심 판정 <code>>=0.85</code>

지표별 산출물 포맷

각 평가 실행은 JSONL로 저장한다.

```
{
  "runId": "autoqa-2026-04-26-stage1",
  "buildId": "game-build-1234",
  "metricId": "M05",
  "score": 0.94,
  "decision": "pass",
  "evidenceMode": "consensus-backed",
  "observations": {
    "logEvent": "QuestCompleted",
    "videoTimestamp": "03:14.2",
    "ocrText": "Quest Complete"
  },
  "retrieval": {
    "documents": ["qa-checklist-stage1.md"],
    "graphPath": ["quest:stage1", "requires", "ui:completion_banner"]
  },
  "llmJudge": {
    "model": "gemini-video-or-frame-judge",
    "confidence": 0.88
  }
}
```

평가 루프

AutoQA 지표 연구는 다음 루프를 따른다.

1. `metric_program.md` 에 목표, 게임 빌드, 데이터셋, 금지 변경 사항을 기록한다.
2. 기존 데이터셋을 고정한다. 영상, 로그, 매크로, OCR/object 결과를 같은 run에 묶는다.
3. 후보 지표 하나를 추가한다.
4. Genkit evaluator 또는 별도 evaluator로 batch run을 수행한다.
5. 사람 QA 라벨과 비교한다.
6. 개선된 지표만 유지하고, 결과는 append-only로 남긴다.
7. threshold를 바꿔야 하면 같은 세션이 아니라 새 comparison track으로 분리한다.

핵심 불변 조건:

- 같은 비교 세션에서는 영상 샘플링 FPS, OCR 엔진, object detector, LLM judge 모델, 검색 corpus 버전을 바꾸지 않는다.

- threshold 변경은 새 metric version으로 기록한다.
- 실패한 지표도 삭제하지 않고 `discarded` 상태로 기록한다.
- 자동 합격/불합격을 낼 수 없는 경우는 `evidence-only` 로 남긴다.

구현 단계 제안

Phase 1: 연구 데이터셋 고정

- 대표 게임 장면 20개 선정
- 각 장면에 영상, 로그, macro, OCR, object 결과 저장
- 사람 QA 라벨 작성
- 게임 문서와 QA 체크리스트를 FunQA 검색 corpus에 적재

Phase 2: 지표 생성기 작성

- 검색 질의 템플릿 작성
- 검색 결과를 metric candidate JSON으로 변환
- 중복 지표 병합
- evidence 부족 지표를 자동 제외

Phase 3: 영상 분석 evaluator 작성

- native video 모델 또는 frame path 중 하나를 baseline으로 선택
- timestamp 기반 이벤트 추출 prompt 작성
- JSON schema validation 추가
- 빠른 액션 장면은 샘플링 FPS를 별도 track으로 분리

Phase 4: consensus evaluator 작성

- programmatic score 계산
- LLM judge score 계산
- 검색 근거 coverage 계산
- 합의 부족 시 evidence-only 반환

Phase 5: release gate

- 핵심 지표 10개 이상 통과
- 사람 QA 판정과 자동 판정 일치율 ≥ 0.90
- critical false pass 0건
- 모든 지표가 citation, timestamp, observation evidence를 가진다.

Genkit/FunQA 적용 포인트

Genkit은 flow, prompt, model 평가를 dataset과 evaluator로 실행할 수 있고, custom evaluator에서 LLM judge 또는 heuristic logic을 둘 다 사용할 수 있다. 따라서 AutoQA는 다음 구조가 자연스럽다.

- `autoqaMetricGenerationFlow` : 검색 결과에서 후보 지표 생성
- `autoqaVideoObservationFlow` : 영상/프레임에서 timestamp 이벤트 추출
- `autoqaMetricEvaluationFlow` : 지표 계산과 consensus 판정
- `custom/autoqaHumanAgreementEvaluator` : 사람 라벨과 자동 판정 일치율 계산
- `custom/autoqaEvidenceCoverageEvaluator` : 각 판정의 검색 근거, timestamp, observation 존재 여부 검사

기존 `/v1/search` 와 `/v1/rag/inspect` 는 AutoQA 검색 근거와 지표 디버깅 화면의 기반으로 재사용할 수 있다.

참고 자료

- Gemini API Video Understanding: <https://ai.google.dev/gemini-api/docs/video-understanding>
- OpenAI Cookbook, video understanding with extracted frames: https://developers.openai.com/cookbook/examples/gpt_with_vision_for_video_understanding
- Genkit Evaluation: <https://genkit.dev/docs/go/evaluation/>
- OpenAI Graders guide: <https://developers.openai.com/api/docs/guides/graders>
- FunQA Consensus RAG V1: `docs/spec/funqa-consensus-rag-v1.md`
- FunQA RAG platform note: `knowledge/wiki/reports/funqa-rag-platform.md`
- Validator evidence: `study/autoqa-game-evaluation-validator-evidence.md`